LMU LUDWIG-MAXIMILIANS-UNIVERSITÄT MÜNCHEN
LMU MUNICH SCHOOL OF MANAGEMENT
INSTITUT FÜR RECHNUNGSWESEN UND WIRTSCHAFTSPRÜFUNG

Empirical Research in Accounting, Auditing and Capital Markets
Tutorial 1 · Reproducible Workflow



Winter semester 2026/27 · M.Sc. / MBR

Dr. Jan Seitz

Room 316 and Zoom, 22 October 2026

TUTORIAL 1 · THURSDAY, 22 OCTOBER 2026, 16:00–20:00

Reproducible Research Workflow

I. Why we are here: an evidence lab
II. Reproducible research
III. R, RStudio and R Markdown
IV. Git and GitHub — version control made simple
V. R basics: from student cities to Compustat
VI. Coding with AI — responsibly
VII. Wrap-up and next steps



Today at a glance — slides and R Markdown go hand in hand

Time	Block	What you do	R Markdown
16:00	I · Why we are here	Tutorial as evidence lab; the Lab assignment	—
16:20	II · Reproducible research	Why results must be re-runnable — by others and by you	—
16:40	III · R, RStudio, R Markdown	Projects, paths, knitting	Part 1
17:10	Break	10 minutes	
17:20	IV · Git and GitHub	Set up Git, fork the course repo, first commit and push	Part 2
18:15	V · R basics	Student cities, objects, data frames, importing Compustat	Parts 3–5
19:05	Break	10 minutes	
19:15	VI · Coding with AI	What AI does well, where it fails; bug hunt	Part 6
19:50	VII · Wrap-up	Checklist, homework, outlook on Tutorial 2	Part 7

▶️ **IN RSTUDIO** **Open Session_01_student.Rmd now — we fill it in together, part by part.**

© Jan Seitz, Ludwig-Maximilians-Universität München

One question runs through every tutorial

“How do I know that this analysis is correct and informative?”

Lecture: a research workshop

- Research questions and the credibility of evidence
- Anchor and frontier papers, roles, discussion
- Toolkit: WHAT/WHY/HOW, Libby boxes, DAGs, validity

Tutorial: an empirical methods lab

- Opens the black box: how is evidence actually produced?
- R/RStudio, R Markdown, reproducible projects
- Data checks, construct validation, diagnostics, simulations
- AI-assisted coding — always independently verified

A successful run does not establish a correct design. Code that runs is not code that is right.

© Jan Seitz, Ludwig-Maximilians-Universität München

From replication to the Research Design & Evidence Lab

Lab component	What you produce	Tutorial
1 · Question and claim	X, Y, population, claim type	3
2 · Design reconstruction	Libby boxes, DAG, assumptions	3, 4
3 · Evidence audit	What each table can and cannot show	3, 4
4 · Assumption challenge and AI audit	One R diagnostic or simulation; audit an AI critique or code product	1–5
5 · Redesign	Stronger design; “what would change my mind?”	4, 5

Replication → Lab

- Then: reproduce a table as closely as possible
- Now: reconstruct the design, **challenge a key assumption**, test it in R
- AI-positive — but every AI claim is checked

30 Nov paper approved · **14 Dec** AI product assigned · **5 Feb** submission

Today builds the foundation for component 4: a reproducible notebook with independent checks.

Source: Course syllabus WS 2026/27, Section 6 (Assignment 3).

© Jan Seitz, Ludwig-Maximilians-Universität München

Five tutorials, one workflow

Date	Tutorial	Focus	Lecture link
Thu 22 Oct	1 · Reproducible research workflow	R/RStudio, project structure, Git/GitHub; responsible AI-assisted coding	Kick-off toolkit
Thu 29 Oct	2 · Data preparation and verification	Identifiers, merges, missingness, duplicates, assertions, auditing generated code	Preparers ↔ Users
Thu 5 Nov	3 · Measurement and construct validity	Descriptives, visualisations, proxies, anchor-paper design exercise	Intermediaries
Thu 26 Nov	4 · Causal inference in practice	Estimands, DAGs, controls, fixed effects, diff-in-diff, diagnostics	Assurance & Enforcement
Fri 27 Nov	5 · Lab workshop	One focused simulation or diagnostic; independent code checks; AI audit preview	Rule-makers
End Jan 2027	Q&A on demand	Support for the Lab and the proposal	Poster workshop

Tutorials in Room 316 and on Zoom. Materials: Moodle + our GitHub repository (today you will connect to it).

© Jan Seitz, Ludwig-Maximilians-Universität München

When nobody can re-run your analysis, nobody can trust it

Reinhart & Rogoff (2010)

- Influential claim: growth drops sharply when public debt exceeds 90% of GDP
- 2013: a graduate student re-ran their spreadsheet — rows were left out of an average, plus other choices
- Corrected numbers: the “cliff” largely disappears
- Found only because the original file could be shared and re-run

Bao et al. (2020), JAR

- Machine-learning model to detect accounting fraud
- Code and data steps were posted on GitHub
- Other researchers re-ran it and questioned key results — up to the question “Should Bao et al. (2020) be retracted?” (Gow, SSRN)
- The debate was possible because the work was reproducible

ROOM AND ZOOM CHAT: WHAT IS THE MOST LIKELY ERROR IN YOUR OWN FUTURE THESIS CODE? ONE WORD.

Errors are normal. Hidden errors are the problem. Reproducibility makes errors findable.

Sources: Herndon, Ash & Pollin (2014), Cambridge Journal of Economics 38(2); Bao et al. (2020), JAR 58(1); Gow (SSRN) “Should Bao et al. (2020) be retracted?”.

© Jan Seitz, Ludwig-Maximilians-Universität München

Reproducible, replicable, robust — what do we mean?

	Same data	Different data
Same analysis	**Reproducible** same result when re-run	**Replicable** holds in a new sample
Different analysis	**Robust** holds under other reasonable choices	**Generalisable** holds in other settings

Why it matters for you

- Your future self is your most frequent co-author
- Supervisors and referees ask: “Can you show me how you got this?”
- Journals (JAR, TAR, JAE …) increasingly require code and data descriptions

“Reproducibility is like brushing your teeth. It is good for you, but it takes time and effort. Once you learn it, it becomes a habit.”

Baker (2016, p. 454)

Minimum standard in this course: anyone with your project folder can re-run your notebook and get the same numbers.

Matrix following The Turing Way Community (2022), The Turing Way: A handbook for reproducible, ethical and collaborative research.

© Jan Seitz, Ludwig-Maximilians-Universität München

From file chaos to a research project

Typical desktop

```
Desktop/
  analysis.R
  analysis_new.R
  analysis_final.R
  analysis_final_v2_JAN.R
  compustat (1).csv
  compustat_clean_REALLY.csv
  Untitled1.Rmd
# which one produced Table 2?
```



One project = one folder

```
EmpRes-Tutorial/
  EmpRes-Tutorial.Rproj  # double-click to open
  README.md              # what is where
  .gitignore             # what Git ignores
  data/                  # raw data, read-only
  session_01/            # one folder per session
    Session_01_student.Rmd
  output/                # everything code creates
```

Four habits: one project folder · code instead of clicks · raw data is never edited · every change is versioned.

© Jan Seitz, Ludwig-Maximilians-Universität München

R is the engine, RStudio is the cockpit

R

- Programming language for statistics
- Does the actual computation

RStudio

- Integrated development environment (IDE)
- Editor, console, environment, Git and the Posit Assistant in one window

Pane	What it is for
Source (top left)	Your scripts and R Markdown files — the permanent record
Console (bottom left)	Try things out; nothing typed here is saved
Environment / Git (top right)	Objects in memory; the Git pane once Git is set up
Files / Help / Viewer (bottom right)	Project files, help pages (?mean), knitted output

Shortcut (Windows · Mac)	Action
Ctrl+Enter · Cmd+Return	Run current line or selection
Ctrl+Alt+I · Cmd+Option+I	Insert a new code chunk

Rule of thumb: if it matters, it goes in the Source pane — never only in the Console.

© Jan Seitz, Ludwig-Maximilians-Universität München

RStudio Projects: never type C:/Users/… again

Absolute path — breaks on every other laptop

```
read_csv("C:/Users/seitz/Documents/
  01_Lehre/compustat.csv")
# Error: file does not exist
```

Relative path — runs everywhere

```
library(here)
read_csv(here("data", "compustat.csv"))
# here() = folder with the .Rproj file
```

What an RStudio Project (.Rproj) does for you

- Sets the working directory to the project folder automatically — no setwd()
- Keeps one RStudio window per project, with its own history and open files
- Enables the Git pane when the folder is a Git repository

▶️ **IN RSTUDIO** — **Part 1 · Chunks 1.1–1.2: where am I? getwd() vs. here()**

© Jan Seitz, Ludwig-Maximilians-Universität München

R Markdown: text, code and results in one document

Three building blocks

YAML header
title, author, output format

Text in Markdown
headings, bold, lists

Code chunks
```{r name} ... ```

Source view

````
---
title: "Tutorial 1"
author: "Your Name"
output: html_document
---

## Student cities
Munich has **many** students.

```{r cities}
cities <- c("Munich", "Espoo")
length(cities)
```
````

Knit = run + render

- Runs every chunk **from scratch** in a clean session
- If it knits, it is reproducible on your machine
- Output: HTML (default today), PDF, Word
- Shortcut: Ctrl+Shift+K

▶️ **IN RSTUDIO** — **Part 1 · Exercise 1.3: put your name in the YAML header and knit the document**

© Jan Seitz, Ludwig-Maximilians-Universität München
````

Markdown in one slide

You write	You get
`# Heading 1` · `## Heading 2`	Section headings
`**bold**` · `*italic*`	**bold** · *italic*
`- item`	Bullet list
`[LMU](https://www.lmu.de)`	Link
`$\beta_1$` · `$$y = a + bx$$`	Inline and block formulas
`> quote`	Block quote

Chunk option	Effect
`echo = FALSE`	Hide code, show result
`eval = FALSE`	Show code, do not run it
`include = FALSE`	Run, show nothing (setup)
`message = FALSE`	Suppress package messages
`error = TRUE`	Keep knitting despite an error

Form follows function: headings that mirror your analysis steps make your notebook auditable.

R Markdown cheat sheet: https://rstudio.github.io/cheatsheets/ · Book: Xie, Allaire & Grolemund, R Markdown: The Definitive Guide (bookdown.org/yihui/rmarkdown).

© Jan Seitz, Ludwig-Maximilians-Universität München

Why version control? Because “final_v2” is not a system

1

Time machine

Every saved state (commit) can be restored. Go back to the version that produced last week’s table.

2

Change log

See what changed, when, and why — line by line. Your research diary for free.

3

Backup and sync

GitHub keeps a copy in the cloud. Laptop broken? Clone and continue.

4

Collaboration

Co-authors work on the same files without e-mailing versions around.

5

Safety net for AI

An AI edited 40 lines? The diff shows every change — keep or undo.

Git is like “Track Changes” for a whole project folder — plus a save button that never overwrites.

Standard tool in data-driven research, open science and every company that writes code.

© Jan Seitz, Ludwig-Maximilians-Universität München

Git and GitHub are not the same thing

Git

- Free program on **your laptop**
- Records snapshots (commits) of your project folder
- Works offline
- Analogy: the save-game system of a video game

GitHub

- Website that **hosts** Git repositories in the cloud
- Backup, sharing, collaboration, issues, project pages
- Free accounts; extras for students (GitHub Education)
- Analogy: the cloud where save games are stored and shared



Today you need both: Git does the work locally, GitHub is where the course material lives.

© Jan Seitz, Ludwig-Maximilians-Universität München

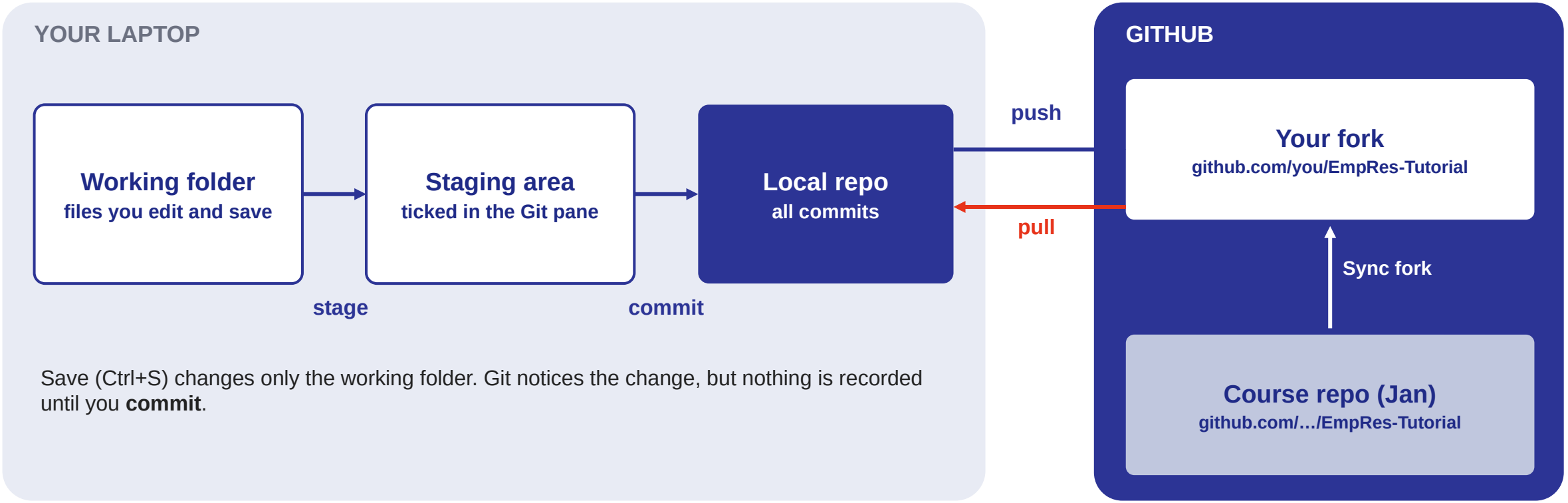
Seven words you need — and nothing more

Word	What it means	Everyday analogy	In RStudio
Repository (repo)	A project folder tracked by Git	A binder with its full edit history	The project folder
Commit	A saved snapshot with a short message	A dated photo of your desk + caption	Commit button
Push	Upload your new commits to GitHub	Upload photos to the cloud	Green arrow ↑
Pull	Download new commits from GitHub	Sync the cloud to your laptop	Blue arrow ↓
Clone	Download a repo from GitHub for the first time	Copy the whole binder once	New Project → Version Control
Fork	Your own copy of somebody else's repo on GitHub	Photocopy of my binder that you may write in	on github.com
.gitignore	List of files Git must never track	"Do not photograph" sticker	text file in the project

Daily life needs only three of them: commit, pull, push.

© Jan Seitz, Ludwig-Maximilians-Universität München

The big picture: where your files travel



Save (Ctrl+S) changes only the working folder. Git notices the change, but nothing is recorded until you **commit**.

Clone once. Afterwards: pull → work → commit → push. That is 95% of Git.

Nothing reaches GitHub without a push — and nothing reaches your laptop without a pull.

© Jan Seitz, Ludwig-Maximilians-Universität München

One-time setup (15 minutes, together)

1 Install Git

Windows: git-scm.com → defaults are fine. Mac: run `xcode-select --install` in the Terminal.

2 Create a GitHub account

github.com — use a professional user name. Students: GitHub Education gives free extras.

3 Tell Git who you are

```
usethis::use_git_config(user.name = "Jane Doe",
user.email = "jane@…")
```

4 Give RStudio a key to GitHub

`usethis::create_github_token()` → copy the token → `gitcreds::gitcreds_set()` → paste.

5 Check RStudio

Tools → Global Options → Git/SVN: path to git is filled in. Restart RStudio.

6 Diagnose

`usethis::git_sitrep()` — reads like a health report. Green = good.

▶️ **IN RSTUDIO** — **Part 2 · Chunks 2.1–2.3 contain exactly these commands (run them in the Console, once)**

Detailed guide: Bryan, J., Happy Git and GitHub for the useR (happygitwithr.com).

© Jan Seitz, Ludwig-Maximilians-Universität München

Get the course material: fork once, clone once

On github.com (browser)

1. Open the course repository (link on screen, in the Zoom chat and on Moodle)
2. Click **Fork** (top right) → **Create fork**. Now you have your own copy: github.com/YOU/EmpRes-Tutorial
3. In your fork: green **<> Code** button → copy the HTTPS URL
4. Later, to get my new material: in your fork click **Sync fork** → **Update branch**

In RStudio

1. File → New Project → **Version Control** → **Git**
2. Paste the URL. Choose a parent folder that is **not** inside OneDrive/Dropbox/iCloud
3. Create Project → RStudio downloads everything and opens EmpRes-Tutorial.Rproj
4. The **Git** tab appears in the top-right pane. Done!

▶️ **IN RSTUDIO** — **Part 2 · Exercise 2.4: fork + clone, then open session_01/Session_01_student.Rmd**

© Jan Seitz, Ludwig-Maximilians-Universität München

The Git pane in RStudio — all buttons you need

Button	What it does
Diff	Shows what changed: green = added, red = removed
Commit	Opens the review window: tick files, write a message
Pull ↓ (blue)	Download new commits from GitHub
Push ↑ (green)	Upload your commits to GitHub
History (clock)	All snapshots — who, when, what
Revert	Throw away unsaved-to-Git changes (careful!)

Status	Meaning
M (blue)	Modified since the last commit
? (yellow)	New file Git does not track yet
A (green)	Added — staged for the first time
D (red)	Deleted

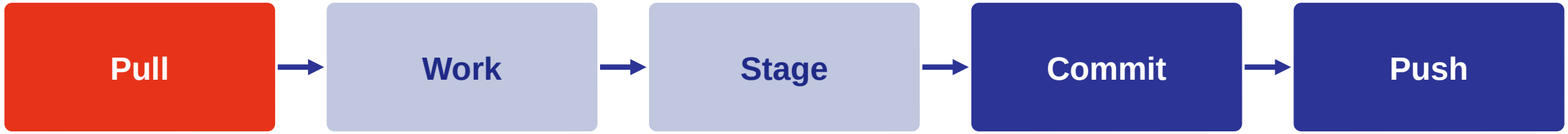
Good commit messages

✓ “Add import checks for compustat”
× “stuff”, “update”, “asdf”

A commit message completes the sentence “If applied, this commit will …”

© Jan Seitz, Ludwig-Maximilians-Universität München

The daily loop: pull → work → commit → push



Start of every session: get the newest version

Edit, run chunks, knit, save

Tick the files that belong together

Snapshot with a clear message

Upload — now it is safe on GitHub

Commit small and often

- One logical step = one commit (e.g., “import data”, “add checks”)
- Commit before you let an AI change your code

Before you push: pull

- If GitHub has something new, Git refuses your push (“rejected”)
- Fix: Pull, then Push again

Commit = save for yourself. Push = save for the world (and your future laptop).

© Jan Seitz, Ludwig-Maximilians-Universität München

What never goes to GitHub: the .gitignore file

Keep out of the repository

- **Licensed data** — Compustat, CRSP, WRDS downloads (licence violation if public!)
- **Personal data** — names, e-mails, survey responses
- **Secrets** — passwords, API keys, GitHub tokens
- **Big or generated files** — GitHub blocks files > 100 MB; outputs can be re-created
- **Clutter** — .Rhistory, .RData, .Rproj.user/

Our .gitignore

```
# .gitignore of our course project
.Rproj.user/
.Rhistory
.RData

# data come from Moodle, never from GitHub
data/*
!data/README.md

# everything the code creates
output/*
*.html
```

Public repository = public. Check before every push: would I put this file on a poster?

© Jan Seitz, Ludwig-Maximilians-Universität München

When Git complains: merge conflicts are normal

What a conflict looks like in your file

```
<<<<<<< HEAD
cities <- c("Munich", "Espoo")      # your version
=======
cities <- c("Munich", "Stockholm")  # GitHub's
version
>>>>>>> origin/main
```

How to solve it

1. Do not panic — nothing is lost
2. Open the file, keep the right lines, delete the markers <<<, ===, >>>
3. Save → stage → commit → push
4. Stuck? Copy your file somewhere safe, re-clone, paste it back

How to avoid them in this course: pull before you start · only edit your own Rmd files · never edit files I might update (README, slides)

Git never silently overwrites your work — it stops and asks you. That is a feature.

© Jan Seitz, Ludwig-Maximilians-Universität München

Your turn: first commit and push

1. Open session_01/Session_01_student.Rmd in your project
2. Put your name in the YAML header, save, knit
3. Git pane: tick Session_01_student.Rmd → Commit
4. Message: “Add my name to tutorial 1” → Commit
5. Push ↑ — enter nothing; the token does the work
6. Open your fork on github.com — is your commit there?

ROOM: THUMBS UP · ZOOM: TYPE “PUSHED” WHEN YOUR COMMIT IS ON GITHUB — OR SHOW US YOUR ERROR MESSAGE.

▶️ **IN RSTUDIO** — **Part 2 · Exercise 2.5: first commit and push (the HTML file is ignored — check why in .gitignore)**

© Jan Seitz, Ludwig-Maximilians-Universität München

R in five minutes: objects, assignment, functions

Chunks 3.1–3.2

```
1 + 1                 # R as a calculator
5 * (4 - 1)^2
sqrt(log(10))

x <- 5                # assign: 'x gets 5'
X <- 6                # different object!
x^2

class(42)             # "numeric"
class("Munich")       # "character"
class(TRUE)           # "logical"
class(42L)            # "integer"
```

Operator	Meaning
`<-`	assign (shortcut Alt + -)
`== !=`	equal / not equal
`< >=`	less than / greater or equal
`& | !`	and / or / not
`#`	comment — explain why, not what
`?mean`	open the help page

Decimal separator is a point: `42.56`. R is case-sensitive: x ≠ X.

▶️ IN RSTUDIO

Part 3 · Chunks 3.1–3.3: calculator, objects and data types, logical operators

© Jan Seitz, Ludwig-Maximilians-Universität München

Our running example: where you study

University	City	Country	Students
LMU Munich	Munich	Germany	8
Aalto University	Espoo	Finland	1
Stockholm School of Economics	Stockholm	Sweden	1
University of Mannheim	Mannheim	Germany	1
University of Eastern Finland	Joensuu	Finland	1
Exchange / second city	Vienna	Austria	+1
Exchange / second city	Shanghai	China	+1

Course party drinks

- Munich — Helles beer / Spezi
- Espoo — Lonkero / Jaffa
- Stockholm — Akvavit / Julmust
- Mannheim — Riesling Schorle / Apfelschorle
- Joensuu — Koskenkorva / Kotikalja
- Vienna — Spritzer / Almdudler
- Shanghai — Huangjiu / Suanmeitang

Small data you know by heart are perfect for checking code: you can see immediately when a result is wrong.

UEF has campuses in Joensuu and Kuopio; we use Joensuu.

© Jan Seitz, Ludwig-Maximilians-Universität München

Vectors: one type, many values

Chunk 4.1–4.2

```
cities <- c("Munich", "Espoo", "Stockholm",
            "Mannheim", "Joensuu", "Vienna", "Shanghai")
inhabitants <- c(1510000, 320000, 990000,
                 315000, 78000, 2000000, 24900000)
names(inhabitants) <- cities

length(cities)                    # 7
inhabitants[2]                    # by position
inhabitants["Vienna"]             # by name
inhabitants[inhabitants > 1e6]    # by condition
sum(inhabitants) / 1e6            # in millions
```

Function	Returns
`c()`	combine values into a vector
`length()`, `sum()`, `mean()`	size, total, average
`min()`, `max()`, `sort()`	extremes, ordered
`seq(1, 10, 2)`, `1:10`	sequences
`rep("beer", 3)`	repetitions

A vector has exactly one data type. Mix numbers and text and everything becomes text.

▶️ IN RSTUDIO

Part 4 · Chunks 4.1–4.2 + Task A: which of “your” cities have more than 1 million inhabitants?

© Jan Seitz, Ludwig-Maximilians-Universität München

Factors and matrices — and the classic matrix trap

Chunks 4.3–4.4

```
country <- factor(c("Germany", "Finland", "Sweden",
  "Germany", "Finland", "Austria", "China"))
table(country)

m <- cbind(inhabitants, students = c(8,1,1,1,1,1,1))
class(m[, "inhabitants"])   # numeric

m2 <- cbind(m, sea = c("NO","YES","YES",
                       "NO","NO","NO","YES"))
m2["Munich", ]
sum(m2[, "inhabitants"])     # Error!
```

What happened?

- A matrix — like a vector — has **one** data type
- The text column “sea” turned all numbers into text: "1510000"
- R does not warn you when it converts — it only fails later

How do I know?

- Check types: `class()`, `str()`, `typeof()`
- Mixed types belong in a **data frame** (next slide)

▶️ **IN RSTUDIO** — **Part 4 · Chunks 4.3–4.4 + Task B: find the trap with class() and fix it**

© Jan Seitz, Ludwig-Maximilians-Universität München

Lists and data frames: the workhorses

Chunk 4.5 — list

```
party <- list(title  = "Course party",
              cities = cities,
              drinks = c(Munich = "Helles", …))
party$drinks["Munich"]
```

Chunk 4.6 — data frame

```
df_cities <- data.frame(city = cities,
  inhabitants = inhabitants, country = country)
str(df_cities)          # one type per column
subset(df_cities, inhabitants < 1e6)
```

Object	Shape	Mixed types?
Vector	1 dimension	No
Factor	1 dimension, labelled	No
Matrix	rows × columns	No
List	anything in a bag	Yes
Data frame	rows × columns	Yes, per column

Tidy data: every column a variable, every row an observation, every cell one value.

▶️ **IN RSTUDIO** — **Part 4 · Chunks 4.5–4.6 + Task C: party planning — how many drinks of each kind do we need?**

© Jan Seitz, Ludwig-Maximilians-Universität München

Packages and real data: importing Compustat

Chunks 5.1–5.3

```
# install.packages("tidyverse")  # once per laptop
library(tidyverse)                # every session
library(here)

cs   <- read_csv(here("data", "compustat.csv"))
cs_b <- read.csv(here("data", "compustat.csv"))
cs_g <- read_csv2(here("data",
                  "compustat_German_format.csv"))

head(cs$gvkey, 2)     # "001004" "001004"
head(cs_b$gvkey, 2)   # 1004 1004   <- zeros gone!
problems(cs)          # 94 values set to NA
```

File	Sep.	Decimal	Function
compustat.csv	,	.	read_csv()
…German_format	;	,	read_csv2()

Two silent surprises

- **gvkey** is a code, not a number: read as a number, “001004” becomes 1004 — later merges fail
- **Type guessing** from the first 1,000 rows: 94 cells silently become NA
- Fix: `col_types = cols(gvkey = "c")`, `guess_max = Inf`

▶️ IN RSTUDIO

Part 5 · Chunks 5.1–5.3: put the Moodle files into data/, import three ways, compare

© Jan Seitz, Ludwig-Maximilians-Universität München

Five checks after every import

Size

Rows and columns as expected?

```
dim(cs)
```

Types

Numbers numeric, codes text, dates dates?

```
glimpse(cs)
```

Keys

One row per firm-year? Here: 57,276 keys for 63,393 rows — why?

```
n_distinct(cs$gvkey, cs$fyear)
```

Ranges

Negative assets? Missing years? Two currencies?

```
summary(cs$at)
```

Known value

Matches a number you can look up?

Apple FY2014: at = 231,839 ($m) — check the 10-K

Turn expectations into assertions — the knit stops if one is violated

```
stopifnot(nrow(cs) == nrow(cs_g), all(nchar(cs$gvkey) == 6), all(cs$at >= 0, na.rm = TRUE))
```

▶️ **IN RSTUDIO** — **Part 5 · Chunk 5.4 + Task D: write three assertions for the Compustat data**

© Jan Seitz, Ludwig-Maximilians-Universität München

AI is part of this course — explicitly

What AI coding assistants do well

- Explain unfamiliar code line by line
- Draft boilerplate: imports, plots, tables
- Translate between languages (Stata → R)
- Read error messages and suggest fixes
- **Review code bases and flag likely bugs** — a tireless second pair of eyes
- Write tests and assertions you forgot

Our tool: the Posit Assistant in RStudio

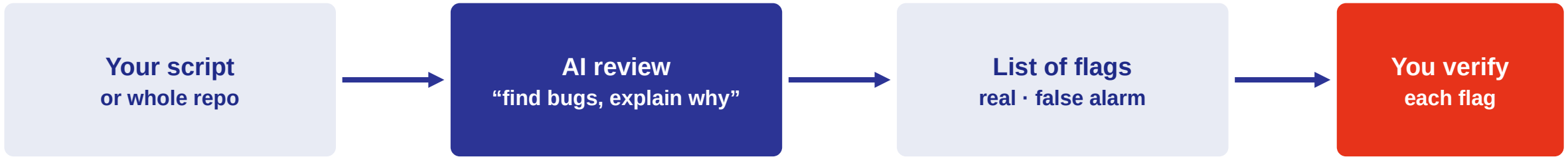
- Chat assistant built into RStudio — no copy-paste between windows
- Sees your live R session: objects, data, errors
- Can explain, write, debug and run code for you
- Because it sees your session, it also sees loaded data — think before you ask

You remain responsible for every claim, citation, output and inference. (Syllabus, Section 5.4)

Posit (2026), "Introducing AI in RStudio", posit.co/blog/introducing-ai-in-rstudio.

© Jan Seitz, Ludwig-Maximilians-Universität München

AI as bug hunter: reviewing a code base



Type of error	Example	AI usually …
Syntax / runtime	missing bracket, wrong function name	finds it (and so does R)
Silent logic error	= instead of ==, missing `na.rm`, off-by-one lag	often finds it — if you ask
Data error	duplicates, wrong units (thousands vs. millions), lost leading zeros	finds it only if it sees the data structure
Design error	wrong control group, look-ahead bias, construct mismatch	often misses it — needs your institutional knowledge

AI finds candidates, not truths. Every flag is a hypothesis you test.

© Jan Seitz, Ludwig-Maximilians-Universität München

The problem: plausible is not the same as correct

Hallucinations

Functions, arguments or packages that do not exist — or citations that were never written.

Silent errors

Code runs, output looks fine, numbers are wrong. The most dangerous case.

Confident tone

Wrong answers sound exactly as sure as right ones.

Design blindness

Plausible-looking research built on spurious relations and weak institutional logic.

Data protection

The Posit Assistant sees your session — including licensed (Compustat) or personal data.

Skill erosion

If AI writes everything, you cannot judge anything — and Q&A is live, without AI.

You still need to understand every line. If you cannot explain it, you cannot use it.

Ciconte, W. A., Rozario, A. M., & Urcan, O. (2026). Using AI to identify exogenous shocks and conduct archival accounting research. Review of Accounting Studies, 31, 1638–1676.

© Jan Seitz, Ludwig-Maximilians-Universität München

How to verify AI-generated code: four checks

1

Known inputs

Run it on data where you know the answer: our seven cities.

2

Assertions

`stopifnot()` for what must be true: no duplicates, no negative assets.

3

Hand calculation

Recompute one number by hand or in Excel.

4

Independent implementation

Solve it a second way (base R vs. dplyr) — do both agree?

A simple routine with the Posit Assistant

1 Commit your code → **2** ask the assistant → **3** read the diff in the Git pane → **4** run your checks → **5** keep or undo

Git + independent checks = AI-assisted code you can defend.

Course syllabus WS 2026/27, Sections 4.4 and 6 (AI audit protocol: verified · qualified · contradicted · unverifiable).

© Jan Seitz, Ludwig-Maximilians-Universität München

Bug hunt: an “AI-written” script with hidden errors

1 Round 1 · by hand · 10′

- Read the script in chunk 6.1
- Run it — what looks wrong?
- Mark every bug with `# BUG:` and a reason
- Use our known city data to check

2 Round 2 · with AI · 10′

- Ask the Posit Assistant in RStudio to review chunk 6.1
- Prompt: “Review this R code for bugs. For each, explain why.” Do not let it fix the code yet
- Compare with your list

3 Round 3 · debrief · 5′

- Which bugs did AI find that you missed?
- Which did it miss?
- Did it invent a bug?
- What does that mean for how you use AI?

▶️ **IN RSTUDIO** — **Part 6 · Chunks 6.1–6.2: bug hunt and corrected version with assertions**

© Jan Seitz, Ludwig-Maximilians-Universität München

What you can do now

- ☐ I can open the course project via the .Rproj file and knit an R Markdown
- ☐ I use relative paths with here() — no C:/Users/… in my code
- ☐ Git is installed; RStudio talks to GitHub (usethis::git_sitrep() is green)
- ☐ I have forked and cloned the course repo and pushed my first commit
- ☐ I know the daily loop: pull → work → commit → push
- ☐ I know what must never go to GitHub (licensed data, personal data, secrets)
- ☐ I can create vectors, factors, matrices, lists and data frames — and check their types
- ☐ I can import a CSV and run five checks plus assertions
- ☐ I use the Posit Assistant to draft and review code — and I verify what I use

Today’s result: a reproducible project that you will use for the rest of the semester.

© Jan Seitz, Ludwig-Maximilians-Universität München

Next steps

Tonight	Finish Parts 3–6 if we did not get through them; commit and push
Before Thu 29 Oct	Git setup works (git_sitrep green); your fork contains your knitted Tutorial 1
Thu 29 Oct	Tutorial 2 · Data preparation and verification: identifiers, merges, missingness, duplicates
Before 2 Nov	Paper Brief on Ball & Brown (1968) — template in the syllabus, Appendix A
By Sat 7 Nov	Firm commitment by e-mail to jan.seitz@lmu.de and ISC registration

Optional reading: Happy Git and GitHub for the useR (happygitwithr.com) · R for Data Science, 2e, Ch. 1–7 (r4ds.hadley.nz)

Stuck with Git? Post the output of usethis::git_sitrep() in the Moodle forum.

© Jan Seitz, Ludwig-Maximilians-Universität München

QUESTIONS?

See you on 29 October

Bring a working project and a green git_sitrep().

jan.seitz@lmu.de

